

实践1：宇树G1仿真环境（Isaac Sim/Lab/MuJoCo）搭建与基础功能验证



深蓝学院《人形机器人运动控制》课程第1章

课程链接：<https://www.shenlanxueyuan.com/course/864>

1. 作业目标

- 搭建宇树 G1 人形机器人强化学习环境
- 跑通 Unitree RL Lab 训练与 Sim2Sim 实验
- 为后续运动控制算法开发与验证打好基础

2. 前置准备

2.1 硬件与软件配置要求

- 操作系统：Ubuntu 22.04
- 显卡 (GPU)：NVIDIA RTX 系列 (显存 ≥ 8 GB)
- 显卡驱动：版本 ≥ 525 ，必须与CUDA版本匹配（如果 Ubuntu 22.04.5 内核为 6.8.0-48-generic 或更高，推荐 NVIDIA driver 535.216.01 或更新版本）
- Isaac Sim 5.1.0
- Isaac Lab 2.3.2
- Python 3.11

2.2 环境预检查

在开始安装前，请在终端执行以下命令验证系统基础条件：

代码块

```
1 # 检查 NVIDIA 显卡状态与驱动版本 (确认CUDA版本 $\geq 12.1$ )
2 nvidia-smi
3
4 # 查看 Linux 内核版本
5 uname -r
```

```
6
7 # 查看 Ubuntu 系统发行版本
8 lsb_release -a
```

2.3 安装方式说明

在 Ubuntu 22.04 上，Isaac Sim 支持 pip 安装和二进制安装两种方式。**优先推荐二进制安装**，原因如下：

- 具备直观清晰的目录结构，便于问题定位
- 通过内置的 `python.sh` 实现与 Isaac Lab 的无缝对接
- 更便于管理路径、缓存、扩展及环境变量
- 与官方文档完全一致，减少兼容性问题

2.4 推荐目录结构

为保持环境一致性，建议统一使用以下目录布局：

代码块

```
1 ~/
2 |— isaacsim/           # Isaac Sim 二进制安装目录
3 |— projects/           # 项目目录
4 |   |— IsaacLab/
5 |   |— unitree_rl_lab/
6 |   |— unitree_model/
7 |   |— unitree_ros/
```

2.5 相关文档连接

- Isaac Lab 官方二进制安装文档：
https://docs.robotsfan.com/isaacsim/source/setup/installation/binaries_installation.html
- Isaac Sim 5.1.0 官方下载页：
<https://docs.isaacsim.omniverse.nvidia.com/5.1.0/installation/download.html>
- Isaac Lab 安装总览：
<https://docs.robotsfan.com/isaacsim/source/setup/installation/index.html>
- Unitree RL Lab 官方 README：[https://github.com/unitreerobotics/unitree_rl_lab?](https://github.com/unitreerobotics/unitree_rl_lab?tab=readme-ov-file)
[tab=readme-ov-file](https://github.com/unitreerobotics/unitree_rl_lab?tab=readme-ov-file)

3. 环境搭建

3.1 安装 Isaac Sim（二进制方式）

3.1.1 下载与解压

从官方下载页获取 `Isaac Sim 5.1.0` Linux x86_64 版本的 zip 二进制包：

- <https://docs.isaacsim.omniverse.nvidia.com/5.1.0/installation/download.html>



深蓝学院也提供了百度网盘下载方式，分享文件为：isaac-sim-standalone-5.1.0-linux-x86_64.zip

链接: https://pan.baidu.com/s/1CnaK4OrxpeLTyKuwl7j_ZA 提取码: u7z4

The screenshot shows the NVIDIA Isaac Sim Documentation page for version 5.1.0. The 'Latest Release' section contains a table with the following data:

Name	Version	Release Date	Links
Isaac Sim	5.1.0	October 2025	Linux (x86_64) Linux (aarch64) Windows
Isaac Sim WebRTC Streaming Client	1.1.5	October 2025	Linux (x86_64) Windows macOS (x86_64) macOS (arm64)
Isaac Sim Assets	5.1.0	October 2025	Robots & Sensors Materials & Props Environments Complete (Part 1 of 3) Complete (Part 2 of 3) Complete (Part 3 of 3)

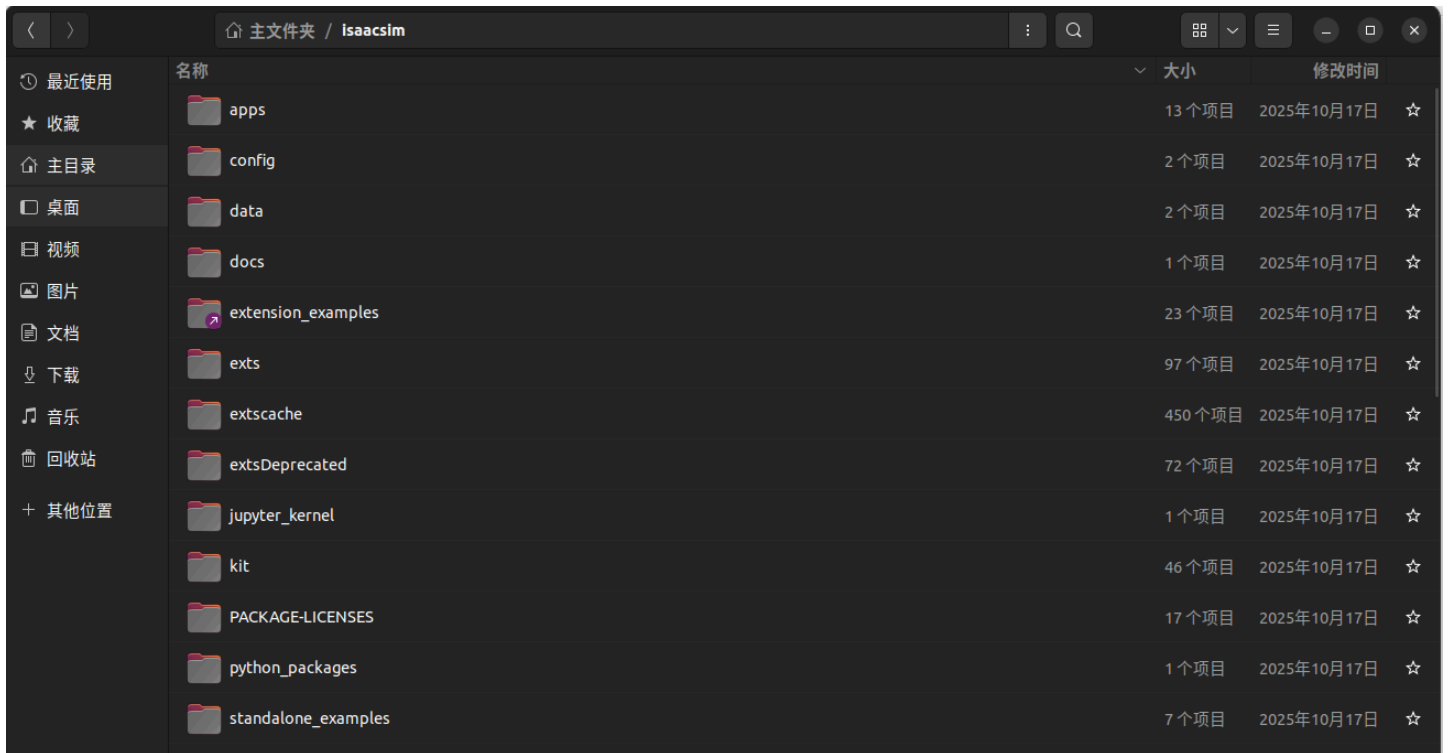
将压缩包解压至 `${HOME}/isaacsim` 目录，解压后目录应包含核心文件：

代码块

```
1  # 创建安装目录
2  mkdir -p ~/isaacsim
3
4  # 进入下载目录并解压
5  cd ~/Downloads
6  unzip 'isaac-sim-standalone-5.1.0-linux-x86_64.zip' -d ~/isaacsim
7
8  # 进入安装目录
9  cd ~/isaacsim
10 # 运行安装后脚本（创建示例扩展的符号链接）
```

```
11 ./post_install.sh
```

解压完成后，目录中应能看到



3.1.2 配置环境变量

将 Isaac Sim 路径写入环境变量，避免每次终端重启重复配置：

代码块

```
1 # 临时导出（当前终端有效）
2 export ISAACSIM_PATH="${HOME}/isaacsim"
3 export ISAACSIM_PYTHON_EXE="${ISAACSIM_PATH}/python.sh"
4
5 # 永久写入 ~/.bashrc（推荐）
6 echo 'export ISAACSIM_PATH="${HOME}/isaacsim"' >> ~/.bashrc
7 echo 'export ISAACSIM_PYTHON_EXE="${ISAACSIM_PATH}/python.sh"' >> ~/.bashrc
8
9 source ~/.bashrc
```

3.1.3 验证安装结果

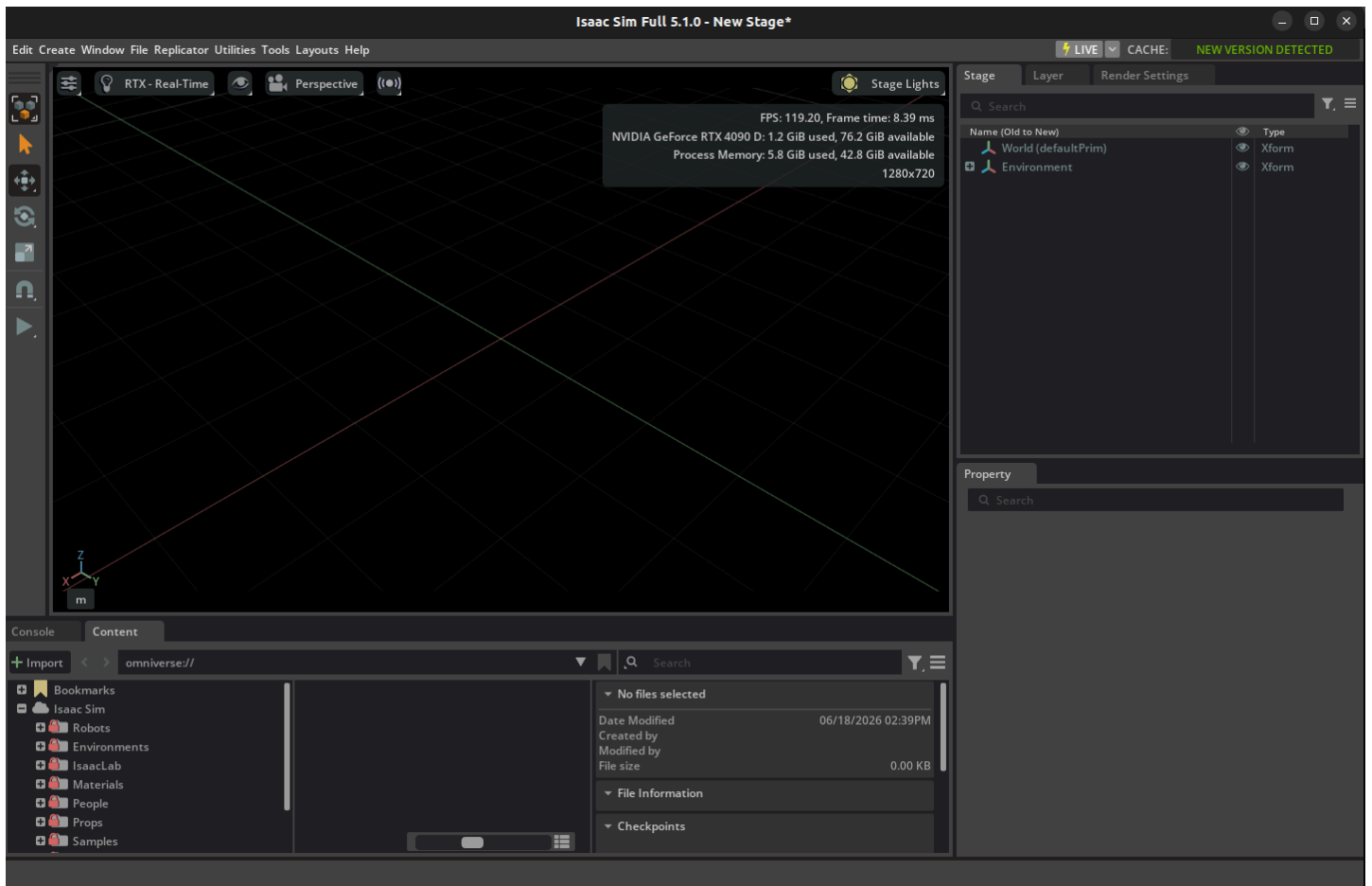
- 验证图形界面

执行以下命令，若能正常启动 Isaac Sim 图形界面，说明基础安装成功：

代码块

```
1 cd ~/isaacsim/
```

```
2 # 启动 Isaac Sim (首次启动会编译着色器, 耗时 5-10 分钟)
3 ./isaac-sim.sh
```



• 验证 Python 环境

执行以下命令，无报错且输出指定文本，说明 Python 环境可用：

代码块

```
1 ${ISAACSIM_PYTHON_EXE} -c "print('Isaac Sim configuration is now complete.')" 
```

3.1.4 旧版本兼容处理

如果你之前用过旧版 Isaac Sim，首次切到 Isaac Sim 5.1.0 后需执行以下命令清理缓存，避免版本冲突：

代码块

```
1 ${ISAACSIM_PATH}/isaac-sim.sh --reset-user
```

3.2 安装 Isaac Lab

3.2.1 克隆源码

代码块

```
1  mkdir -p ~/projects
2  cd ~/projects
3  git clone https://github.com/isaac-sim/IsaacLab.git
4  cd IsaacLab
```

3.2.2 创建软链接

通过软链接将 Isaac Lab 与 Isaac Sim 关联，这是 Isaac Lab 识别 Isaac Sim 环境的关键步骤：

代码块

```
1  cd ~/projects/IsaacLab
2  ln -s ~/isaacsim _isaac_sim
```

验证链接是否生效：

代码块

```
1  ls -l _isaac_sim # 输出应显示 -> /home/[你的用户名]/isaacsim
```

3.2.3 验证安装

代码块

```
1  cd ~/projects/IsaacLab
2  ./isaacsim.sh --help
```

如果命令能正常显示帮助信息，说明 `IsaacLab` 脚本本身工作正常。

```
→ IsaacLab git:(main) ./isaacclab.sh --help

usage: isaacclab.sh [-h] [-i] [-f] [-p] [-s] [-t] [-o] [-v] [-d] [-n] [-c] [-u] -- Utility to
manage Isaac Lab.

optional arguments:
  -h, --help            Display the help content.
  -i, --install [LIB]   Install the extensions inside Isaac Lab and learning frameworks as e
extra dependencies. Default is 'all'.
  -f, --format          Run pre-commit to format the code and check lints.
  -p, --python          Run the python executable provided by Isaac Sim or virtual environme
nt (if active).
  -s, --sim            Run the simulator executable (isaac-sim.sh) provided by Isaac Sim.
  -t, --test           Run all python pytest tests.
  -o, --docker         Run the docker container helper script (docker/container.sh).
  -v, --vscode         Generate the VSCode settings file from template.
  -d, --docs           Build the documentation from source using sphinx.
  -n, --new            Create a new external project or internal task from template.
  -c, --conda [NAME]   Create the conda environment for Isaac Lab. Default name is 'env_isa
acclab'.
  -u, --uv [NAME]      Create the uv environment for Isaac Lab. Default name is 'env_isaACL
ab'.
```

3.3 安装依赖

3.3.1 安装 miniconda

如果尚未安装 conda，推荐安装轻量级的 miniconda

代码块

```
1  # 下载 Miniconda 安装脚本
2  wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
3
4  # 执行安装
5  bash Miniconda3-latest-Linux-x86_64.sh
6
7  # 安装完成后，重新打开终端或执行 source ~/.bashrc 使配置生效
8  source ~/.bashrc
```

3.3.2 Python 依赖

用 Isaac Lab 脚本自动创建环境

```
1 cd ~/projects/IsaacLab
2
3 # 调用 conda 创建一个独立的适用于 Isaac Lab 的 Python 虚拟环境 (默认名称为 env_isaaclab)
4 ./isaac_lab.sh --conda
5
6 # 激活环境
7 conda activate env_isaaclab
8
9 # 设置清华源为默认源
10 pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple
11
12 # 确保当前还在 env_isaaclab 环境中, 安装 Isaac Lab 及强化学习框架
13 ./isaac_lab.sh --install
```

验证安装

代码块

```
1 # 检查环境变量 (确保 Isaac Sim 路径挂载正确)
2 python -c "import isaacsim; print('Isaac Sim imported successfully!')"
3
4 # 检查 rsl_rl (Unitree 训练所需)
5 python -c "import rsl_rl; print('rsl_rl imported successfully!')"
```

3.3.3 系统依赖

代码块

```
1 sudo apt install cmake build-essential
```

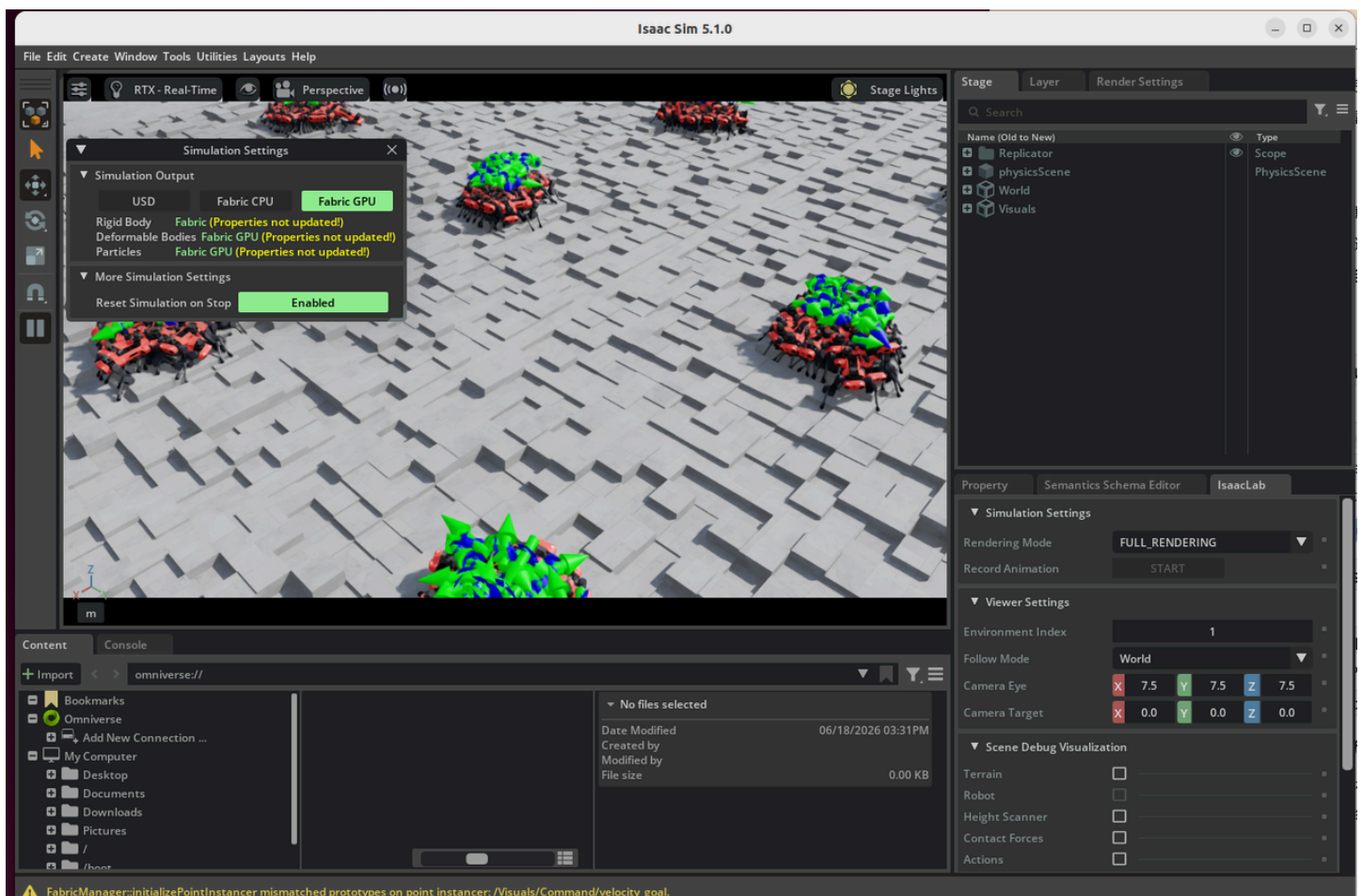
3.4 验证 Isaac Lab

激活 `conda` 环境后直接运行：

代码块

```
1 python scripts/tutorials/00_sim/create_empty.py
2 python scripts/reinforcement_learning/rsl_rl/train.py --task=Isaac-Velocity-
  Rough-AnyMal-C-v0
```

如果这个任务能正常开始训练，则说明 Isaac Lab 环境已具备后续运行 Unitree RL Lab 的基本条件。



💡 常见报错: `No module named 'isaacsim'`

如果出现这个错误, 优先检查:

- `~/projects/IsaacLab/_isaac_sim` 软链接是否存在
- `~/isaacsim/python.sh` 是否存在
- 是否执行过 `conda activate env_isaaclab`
- 是否执行过 `source _isaac_sim/setup_conda_env.sh`

3.5 安装 Unitree RL Lab

3.5.1 克隆仓库

⚠️ Unitree RL Lab 不要放在 IsaacLab 目录内, 避免路径冲突:

代码块

```
1 cd ~/projects
2 git clone https://github.com/unitreerobotics/unitree_rl_lab.git
3 cd unitree_rl_lab
```

3.5.2 安装 Python 包

使用已配置好 Isaac Lab/Isaac Sim 的 Python 环境执行：

代码块

```
1 conda activate env_isaaclab
2 python -m pip install -e source/unitree_rl_lab
```

3.6 配置 Unitree 机器人模型

3.6.1 USD 方式（推荐）

先下载 `unitree_model`：



深蓝学院提供了 unitree_model 百度网盘下载方式，分享文件为：unitree_model

链接: https://pan.baidu.com/s/1kVVv_OkGbzfYZToowk8-g 提取码: gdmw

代码块

```
1 cd ~/projects
2
3 # 安装 Git LFS (需先安装才能下载大文件)
4 git lfs install
5
6 # 克隆模型仓库
7 git clone https://huggingface.co/datasets/unitreerobotics/unitree_model
```

修改模型路径配置文件

```
~/projects/unitree_rl_lab/source/unitree_rl_lab/unitree_rl_lab/assets/robots/unitree.py
```

将UNITREE_MODEL_DIR和UNITREE_ROS_DIR换为实际路径

```
unitree.py
~/projects/unitree_rl_lab/source/unitree_rl_lab/unitree_rl_lab/assets/robots 保存(S)

1 # Copyright (c) 2022-2025, The Isaac Lab Project Developers.
2 # All rights reserved.
3 #
4 # SPDX-License-Identifier: BSD-3-Clause
5
6 """Configuration for Unitree robots.
7
8 Reference: https://github.com/unitreerobotics/unitree_ros
9 """
10
11 import os
12
13 import isaacsim as sim_utils
14 from isaacsim.actuators import IdealPDActuatorCfg, ImplicitActuatorCfg
15 from isaacsim.assets.articulation import ArticulationCfg
16 from isaacsim.utils import configclass
17
18 from unitree_rl_lab.assets.robots import unitree_actuators
19
20 UNITREE_MODEL_DIR = "path/to/unitree_model" # Replace with the actual path to your unitree_model directory
21 UNITREE_ROS_DIR = "path/to/unitree_ros" # Replace with the actual path to your unitree_ros package
22
23
24 @configclass
25 class UnitreeArticulationCfg(ArticulationCfg):
26     """Configuration for Unitree articulations."""
27
28     joint_sdk_names: list[str] = None
29
30     soft_joint_pos_limit_factor = 0.9
31
32
33 @configclass
34 class UnitreeUsdFileCfg(sim_utils.UsdFileCfg):
35     activate_contact_sensors: bool = True
36     rigid_props = sim_utils.RigidBodyPropertiesCfg(
37         disable_gravity=False,
38         retain_accelerations=False,
39         linear_damping=0.0,
40         angular_damping=0.0,
41         max_linear_velocity=1000.0,
42         max_angular_velocity=1000.0,
43         max_depenetration_velocity=1.0,
44     )
45     articulation_props = sim_utils.ArticulationRootPropertiesCfg(
46         enabled_self_collisions=True, solver_position_iteration_count=8, solver_velocity_iteration_count=4
47     )
48
49
Python 2 制表符宽度: 8 第 20 行, 第 41 列 插入
```

代码块

```
1 #打开以下路径的文件
2 ~/projects/unitree_rl_lab/source/unitree_rl_lab/unitree_rl_lab/assets/robots/unitree.py
3
4 # 将username替换为自己的用户名实际路径
5 UNITREE_MODEL_DIR = "/home/username/projects/unitree_model"
6 UNITREE_ROS_DIR = "/home/username/projects/unitree_ros"
```

```
unitree.py
~/projects/unitree_rl_lab/source/unitree_rl_lab/unitree_rl_lab/assets/robots 保存(S)

1 # Copyright (c) 2022-2025, The Isaac Lab Project Developers.
2 # All rights reserved.
3 #
4 # SPDX-License-Identifier: BSD-3-Clause
5
6 """Configuration for Unitree robots.
7
8 Reference: https://github.com/unitreerobotics/unitree_ros
9 """
10
11 import os
12
13 import isaacsim as sim_utils
14 from isaacsim.actuators import IdealPDActuatorCfg, ImplicitActuatorCfg
15 from isaacsim.assets.articulation import ArticulationCfg
16 from isaacsim.utils import configclass
17
18 from unitree_rl_lab.assets.robots import unitree_actuators
19
20 UNITREE_MODEL_DIR = "/home/slxxy/projects/unitree_model" # Replace with the actual path to your unitree_model
21 directory
22 UNITREE_ROS_DIR = "/home/slxxy/projects/unitree_ros" # Replace with the actual path to your unitree_ros package
23
24 @configclass
25 class UnitreeArticulationCfg(ArticulationCfg):
26     """Configuration for Unitree articulations."""
27
28     joint_sdk_names: list[str] = None
29
30     soft_joint_pos_limit_factor = 0.9
31
32
33 @configclass
34 class UnitreeUsdFileCfg(sim_utils.UsdFileCfg):
35     activate_contact_sensors: bool = True
36     rigid_props = sim_utils.RigidBodyPropertiesCfg(
37         disable_gravity=False,
38         retain_accelerations=False,
39         linear_damping=0.0,
40         angular_damping=0.0,
41         max_linear_velocity=1000.0,
42         max_angular_velocity=1000.0,
43         max_depenetration_velocity=1.0,
44     )
45     articulation_props = sim_utils.ArticulationRootPropertiesCfg(
46         enabled_self_collisions=True, solver_position_iteration_count=8, solver_velocity_iteration_count=4
47     )
```

3.6.2 URDF 方式（可选）

若需使用 URDF 格式模型，执行以下命令：

代码块

```
1 cd ~/projects
2 git clone https://github.com/unitreerobotics/unitree_ros.git
```

并在 `unitree.py` 中配置对应 URDF 目录路径。通常仅在需要修改机器人描述或调整导入方式时才需下载此资源。

3.7 验证 Unitree RL Lab

在 `~/projects/unitree_rl_lab` 根目录执行以下命令，验证环境是否完全就绪：

代码块

```
1 python scripts/list_envs.py
```

```
(env_isaac) → unitree_rl_lab git:(main) ✗ python scripts/list_envs.py
+-----+-----+-----+-----+
|                                     Available Environments in Unitree RL Lab                                     |
+-----+-----+-----+-----+
| S. No. | Task Name                               | Entry Point                               | Config                                                                 |
+-----+-----+-----+-----+
| 1       | Unitree-G1-29dof-Velocity               | isaacsim.envs:ManagerBasedRLEnv          | locomotion.robots.g1.29dof.velocity_env_cfg:RobotEnvCfg             |
| 2       | Unitree-Go2-Velocity                    | isaacsim.envs:ManagerBasedRLEnv          | locomotion.robots.go2.velocity_env_cfg:RobotEnvCfg                  |
| 3       | Unitree-H1-Velocity                     | isaacsim.envs:ManagerBasedRLEnv          | locomotion.robots.h1.velocity_env_cfg:RobotEnvCfg                   |
| 4       | Unitree-G1-29dof-Mimic-Dance-102        | isaacsim.envs:ManagerBasedRLEnv          | mimic.robots.g1_29dof.dance_102.tracking_env_cfg:RobotEnvCfg         |
| 5       | Unitree-G1-29dof-Mimic-Gangnam-Style    | isaacsim.envs:ManagerBasedRLEnv          | mimic.robots.g1_29dof.gangnam_style.tracking_env_cfg:RobotEnvCfg     |
+-----+-----+-----+-----+
(env_isaac) → unitree_rl_lab git:(main) ✗
```

4. 作业任务

- 使用默认任务配置 `--task Unitree-G1-29dof-Velocity` 完成一次完整训练过程
- 通过 TensorBoard 记录奖励值等指标曲线
- 使用训练得到的 checkpoint 完成一次 play 验证
- 在 MuJoCo 中完成一次 Sim2Sim 验证

4.1 阅读关键文件

在实践1中，不要求深入修改代码，但要求你能定位几个最关键的入口文件。请重点阅读：

1. source/unitree_rl_lab/unitree_rl_lab/tasks/locomotion/robots/g1/29dof/init.py
2. source/unitree_rl_lab/unitree_rl_lab/tasks/locomotion/robots/g1/29dof/velocity_env_cfg.py
3. source/unitree_rl_lab/unitree_rl_lab/tasks/locomotion/agents/rsl_rl_ppo_cfg.py
4. scripts/rsl_rl/train.py
5. scripts/rsl_rl/play.py
6. scripts/list_envs.py

4.2 启动训练过程

代码块

```
1 python scripts/rsl_rl/train.py --headless --task Unitree-G1-29dof-Velocity
```

终端会输出日志：

```
#####
                        Learning iteration 2/50000
                        #####
                        Computation: 30361 steps/s (collection: 3.163s, learning 0.074s)
                        Mean action noise std: 0.99
                        Mean value_function loss: 0.2045
                        Mean surrogate loss: -0.0095
                        Mean entropy loss: 40.8609
                        Mean reward: -3.64
                        Mean episode length: 47.29
                        Episode_Reward/track_lin_vel_xy: 0.0273
                        Episode_Reward/track_ang_vel_z: 0.0022
                        Episode_Reward/alive: 0.0078
                        Episode_Reward/base_linear_velocity: -0.0046
                        Episode_Reward/base angular velocity: -0.0309
                        Episode_Reward/joint_vel: -0.0185
                        Episode_Reward/joint_acc: -0.0089
                        Episode_Reward/action_rate: -0.1512
                        Episode_Reward/dof_pos_limits: -0.0011
                        Episode_Reward/energy: -0.0008
                        Episode_Reward/joint_deviation_arms: -0.0030
                        Episode_Reward/joint_deviation_waists: -0.0183
                        Episode_Reward/joint_deviation_legs: -0.0233
                        Episode_Reward/flat_orientation_l2: -0.0233
                        Episode_Reward/base_height: -0.0026
                        Episode_Reward/gait: 0.0118
                        Episode_Reward/feet_slide: -0.0099
                        Episode_Reward/feet_clearance: 0.0491
                        Episode_Reward/undesired_contacts: -0.0035
                        Curriculum/terrain_levels: 3.6635
                        Curriculum/lin_vel_cmd_levels: 0.1000
                        Metrics/base_velocity/error_vel_xy: 0.0511
                        Metrics/base_velocity/error_vel_yaw: 0.3485
                        Episode Termination/time_out: 0.1667
                        Episode Termination/base_height: 0.0000
                        Episode Termination/bad_orientation: 46.3750
```

训练过程中会自动保存检查点到 `logs/` 目录。查看日志曲线，新开终端，使用 tensorboard：

代码块

```
1 tensorboard --logdir=logs/Unitree-G1-29dof-Velocity/rsl_rl/latest.pt
```

4.3 训练结果推理验证

代码块

```
1 # 使用最新检查点进行可视化推理
2 python scripts/rsl_rl/play.py --task Unitree-G1-29dof-Velocity
```

成功执行将启动 Isaac Sim 并显示 G1 机器人行走动画。



4.4 Sim2Sim (MuJoCo) 验证

参考: https://github.com/unitreerobotics/unitree_rl_lab?tab=readme-ov-file#4.1

安装系统依赖与 Unitree SDK2

代码块

```
1  # Install dependencies
2  sudo apt install -y libyaml-cpp-dev libboost-all-dev libeigen3-dev libspdlog-
  dev libfmt-dev
3
4  # Install unitree_sdk2
5  git clone git@github.com:unitreerobotics/unitree_sdk2.git
6  cd unitree_sdk2
7  mkdir build && cd build
8  cmake .. -DBUILD_EXAMPLES=OFF # Install on the /usr/local directory
9  sudo make install
```

编译专门用于 Sim2Sim 和真机部署的 G1-29dof 控制器, 编译成功后, 会在 `build` 目录下生成 `g1_ctrl` 可执行文件。

代码块

```
1  # Compile the robot_controller
2  cd unitree_rl_lab/deploy/robots/g1_29dof # or other robots
3  mkdir build && cd build
4  cmake .. && make
```

安装 `unitree_mujoco`, 然后修改仿真配置文件, 编辑

`unitree_mujoco/simulate/config.yaml`, 将以下参数修改为 G1 专用配置:

- Set the `robot` at `/simulate/config.yaml` to `g1`
- Set `domain_id` to 0
- Set `enable_elastic_hand` to 1
- Set `use_joystick` to 1

启动 Sim2Sim 验证流程：

- 步骤 1：启动 MuJoCo 仿真环境

代码块

```
1  # start simulation
2  cd unitree_mujoco/simulate/build
3  ./unitree_mujoco
4  # ./unitree_mujoco -i 0 -n eth0 -r g1 -s scene_29dof.xml # alternative
```

成功启动后会弹出 MuJoCo 窗口，显示 G1 机器人初始姿态。

- 步骤 2：启动 G1 控制器

新开一个终端，启动编译好的控制器：

代码块

```
1  cd unitree_rl_lab/deploy/robots/g1_29dof/build
2  ./g1_ctrl
3  # 1. press [L2 + Up] to set the robot to stand up
4  # 2. Click the mujoco window, and then press 8 to make the robot feet touch
    the ground.
5  # 3. Press [R1 + X] to run the policy.
6  # 4. Click the mujoco window, and then press 9 to disable the elastic band.
```

控制器会自动加载 `unitree_rl_lab/logs/Unitree-G1-29dof-Velocity/rsl_rl/latest.pt` 训练好的最新模型。



5. 提交内容

请提交一份 PDF 报告，建议 3 到 6 页，包含以下内容。

5.1 环境搭建结果

至少给出以下内容中的 3 项：

- Isaac Sim 成功启动截图
- Isaac Lab 场景成功启动截图
- `python scripts/list_envs.py` 的结果截图
- `train.py` 启动训练日志截图
- `play.py` 运行截图
- Sim2Sim实验过程截图

5.2 项目理解部分

请简要说明：

1. Unitree-G1-29dof-Velocity 的任务入口在哪个文件
2. 环境配置在哪个文件
3. PPO 配置在哪个文件
4. 训练入口在哪个文件
5. 默认训练得到的 checkpoint 大致保存在哪类目录

5.3 简短分析

请用自己的话回答：

1. 从 `--task` 到实际环境创建的大致流程是什么
2. 训练配置与播放配置为什么要分开
3. `play.py` 为什么能够自动找到训练得到的 checkpoint

5.4 提交方式

1. 统一通过深蓝学院课程平台作业提交入口上传
2. 提交格式：`.zip` 格式压缩包
3. 压缩包命名：`用户名-实践1.zip`，压缩包大小建议小于100M。

6. 常见问题

6.1 Isaac Sim 能否通过 pip 安装

两种安装方式均支持完整训练流程，但适用场景不同：

- pip 安装：通过 `pip install isaacsim` 命令一键部署，操作简单，适合快速体验或临时环境。但缺点是每个 Conda 环境都需要独立安装（约占用 15GB 空间 / 环境），且多版本切换复杂，缓存清理麻烦。
- 二进制包安装：本指南推荐的方式，实现全局共享，无需为每个 Conda 环境重复安装 Isaac Sim，大幅节省磁盘空间。同时目录结构清晰，便于管理扩展、日志和缓存，支持多版本 Isaac Sim 共存，切换仅需修改软链接即可。

⚠ 注意：即使使用 pip 安装 Isaac Sim，仍需在 Isaac Lab 目录下创建 `_isaac_sim` 软链接，指向 pip 安装的 Isaac Sim 路径（通常为 `~/miniconda3/envs/[环境名]/lib/python3.11/site-packages/isaacsim`）。

6.2 为什么 `_isaac_sim` 软链接如此关键

这是 Isaac Lab 与 Isaac Sim 建立连接的核心机制，其作用远不止“让 Isaac Lab 找到 Isaac Sim”：

1. 统一模块导入路径：Isaac Lab 的所有脚本默认会将 `./_isaac_sim` 目录加入 Python 系统路径，因此只有创建该软链接，才能正常导入 `isaacsim` 及其所有扩展模块（如 `isaacsim.core`、`isaacsim.robotics` 等）。若缺失软链接，会直接抛出 `ModuleNotFoundError: No module named 'isaacsim'` 错误。
2. 解耦安装路径：无论 Isaac Sim 安装在系统的任何位置（全局目录、用户目录、pip 环境内），只需通过软链接指向实际路径，Isaac Lab 无需修改任何代码即可适配，极大提升了环境配置的灵活性。
3. 简化多版本切换：当需要测试不同版本的 Isaac Sim 时，只需删除旧软链接并重新创建指向新版本的链接，无需重新安装 Isaac Lab 或修改环境变量，是多版本开发环境的最佳实践。
4. 兼容官方脚本逻辑：Isaac Lab 提供的 `isaaclab.sh` 脚本、训练模板和扩展工具均基于该软链接设计，遵循此规范可确保所有官方功能正常运行，避免出现路径相关的隐性 bug。

6.3 Isaac Sim 切换版本后启动异常

如果你机器上之前装过旧版本，执行如下命令清理缓存：

代码块

```
1  ${ISAACSIM_PATH}/isaac-sim.sh --reset-user
```

6.4 强化学习库安装失败

若因网络问题导致 `rsl_rl` 或 `rl_games` 安装失败，手动安装：

代码块

```
1  cd ~/projects/IsaacLab/source/extensions
2
3  # 手动安装rl_games
4  git clone https://github.com/isaac-sim/rl_games.git
5  cd rl_games && pip install -e . && cd ..
6
7  # 手动安装rsl_rl
8  git clone https://github.com/leggedrobotics/rsl_rl.git
9  cd rsl_rl && pip install -e . && cd ..
10
11 # 跳过RL库重新安装Isaac Lab
12 cd ~/projects/IsaacLab
13 ./isaacsim.sh --install --skip-rl
```

6.5 Unitree RL Lab 找不到模型文件

- 确认 `UNITREE_MODEL_DIR` 使用绝对路径
- 检查是否已执行 `git lfs install`
- 重新克隆 `unitree_model` 仓库，确保所有文件完整下载